

Teaching Mario: What Works, and How We Test It

Reinforcement learning on real Super Mario Bros, CPU only. Status report and method. github.com/NickNojiri/mario-rl

1. Where the project stands

Piece	State	Evidence
Environment and tooling (WSL, pinned deps, 57 tests)	Working	Tests cover replay memory, advantages, observations, rewards, generated terrain
Tile-grid observation read from game memory	Working	Verified against screenshots on overworld, underground and castle stages
PPO trainer (12 parallel games, ~600 steps/s)	Working	Resumable, snapshots, CSV logs, detached launcher
Evaluation protocol (held-out levels, random baseline)	Working	Same seeds and protocol for every run; uncertainty reported
Procedural level generator inside the real game	Working	Terrain, pipes, tile themes verified solid; flags reachable; enemies kept on the surface
Live viewers (single and multi-game, 1x-8x)	Working	Browser stream, follows training
An agent that generalizes to unseen levels	Not yet	Best is 674 vs 382 random; no flags on held-out levels

2. The scientific approach

The project runs as a loop: state a claim, measure it, keep only what survives measurement. Six rules have done most of the work.

Rule	How it is applied here
1. One primary number, fixed in advance	Held-out distance: mean x_pos over 5 real levels never trained on, 10 episodes each, fixed seeds, random start delays. A random policy (382) anchors the bottom of the scale.
2. Know the noise before comparing	Standard error is measured, not assumed: +-36 with 6 episodes per level, +-25 with 12. Differences under about 2 standard errors are reported as noise, not results.
3. Verify every claim about the game	Nine probe scripts check memory layout and physics against the running emulator: timer expiry kills Mario, enemy speeds and positions, when level columns are written, and a measured jump table.
4. Change one thing at a time, equal budget	Every variant trains from scratch for the same number of steps and gets the same evaluation. The 18-variant sweep and the v3/v4 experiments follow this.
5. Keep the test set clean	Held-out levels are never trained on, never used to pick settings, and generated terrain can never appear in evaluation (any evaluation names its level explicitly, which disables generation).
6. Report negative results and diagnose them	Most ideas failed. Each failure gets a measurement: why it failed, and what it implies.

Instruments built along the way

- **Jump table** (scripted jumps): max distance 3.3 tiles standing, 5.2 walking, 9.8 running; a full jump needs A held 6-8 decisions. This defines what terrain is possible.
- **Pit forensics**: classifies every pit death (walked off, released A early, too slow, took off too early) and measures how long the danger was visible.

- **Behaviour logging:** cause of death, stomps, jumps, LEFT/NOOP usage and every reward component, per episode.
- **Early-warning evaluation:** held-out check at every snapshot, so a run that is going wrong shows it hours before it ends.

3. Every experiment, same measurement

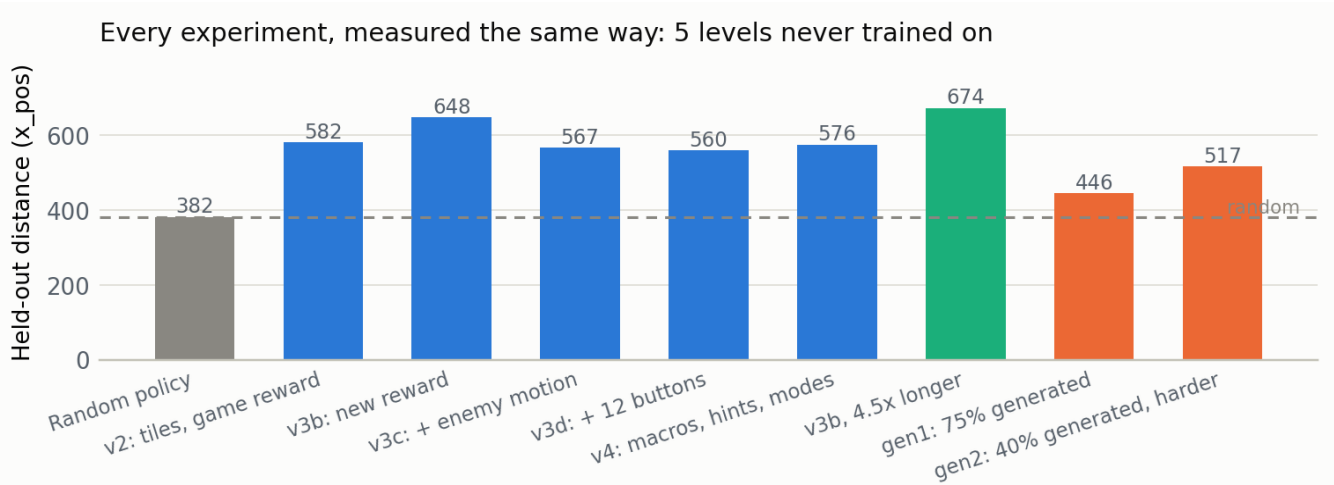


Figure 1. Held-out distance for each configuration. Blue: equal-length (1.72M step) runs. Green: the best overall (same recipe, 4.5x longer). Orange: generated-level runs.

Run	Held-out x_pos	Training x_pos	Training flag rate	Note
Random policy	382	443	0.0%	chance
v2: tiles, game reward	582	774	0.5%	1.72M steps
v3b: new reward	648	924	1.8%	1.72M steps, best short run
v3c: + enemy motion	567	860	1.4%	1.72M steps
v3d: + 12 buttons	560	853	0.5%	1.72M steps
v4: macros, hints, modes	576	830	3.2%	1.72M steps
v3b, 4.5x longer	674	1440	11.8%	7.8M steps, best overall
gen1: 75% generated	446	757	5.0%	7.8M steps
gen2: 40% generated, harder	517	1135	12.3%	7.8M steps

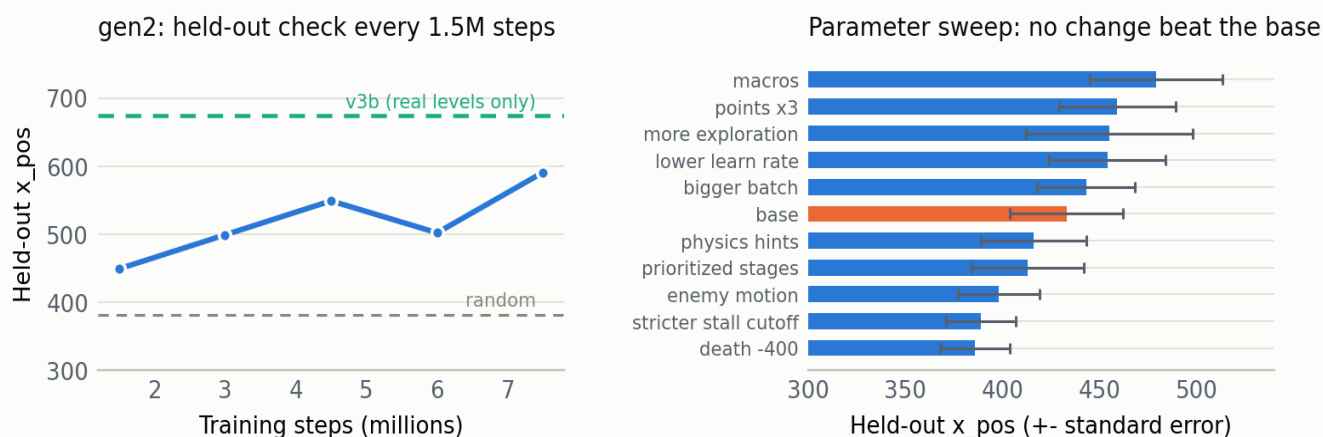


Figure 2. Left: the generated-level run improves steadily on held-out levels and had not flattened when it stopped. Right: 18 one-change variants; every bar overlaps the base within noise.

4. What has actually been learned

Finding	Evidence	Consequence
Reward design mattered more than tuning	New reward: 582 -> 648 held-out. 18 tuning variants: none beyond noise	Stopped tuning; spent effort on the task instead
More training improves known levels, not new ones	4.5x longer: training flags 1.8% -> 11.8%, held-out 648 -> 674	The gap is memorization, not lack of practice
The agent could not see enemy motion	A goomba moves 2 px per decision; 4 stacked 16 px grids hide direction	Added exact enemy positions, speeds and states
Half of pit deaths were walk-offs, not bad jumps	Pit visible ~12 decisions ahead, yet jump probability at the edge was 48%; failed jumps held A 2 steps vs 5 for successes	Added macro jumps (hold A for a fixed time): releasing A early fell from 14% to 1% of pit deaths
Generated levels can be built inside the real game	Terrain written into the collision map; pits, walls, pipes and themes verified solid; flags reachable	Unlimited new layouts with the game's own physics
Training mostly on generated levels backfired	75% generated: 33% of generated levels finished but held-out fell to 446 and randomness collapsed	Generated levels must resemble real ones, and stay a minority of practice
Harder, more realistic generated levels recovered most of it	40% generated, harder terrain, pipes, tile themes, faster enemies: held-out 517, and equal to the best run on 4 of 5 held-out levels; the castle level accounts for the gap	Generated castles (lava, fire bars) are the next piece

5. What is still open

- **No held-out level has ever been finished.** The best agent gets about a fifth of the way through, so the honest claim is 'clearly better than chance, far from playing Mario'.
- **Castles are the weak spot** (held-out 5-4: 320 vs 948). Lava reads as an ordinary pit and fire-bar arms are invisible in the observation.
- **Single seed per configuration.** Reinforcement learning varies between runs; conclusions here rest on one run each, plus the measured evaluation noise.
- **Distance is a proxy.** Levels differ in length, so x_pos is comparable across runs on the same levels, not across levels.

6. Running now, and next

The harder-generator run is continuing to 15M steps, with a held-out check every 2M. Next: generated castle levels, then a like-for-like rematch against the best real-levels-only run at equal steps.

```
python train_ppo.py --preset ppo_gen2 --run-name gen2      # 40% generated, harder, faster
python eval_stages.py runs/gen2/latest.pt                 # held-out + training levels
python -m scripts.sweep --stage a                         # one-change variants, ranked
python -m scripts.pit_forensics runs/gen2/latest.pt       # why it dies
python -m scripts.multi_demo --envs 4 --speed 2 --procgen 0.5 # watch 4 games at 2x
```

Reproducibility. Pinned environment, fixed seeds, config hash in every checkpoint, 57 automated tests, and every figure here traceable to a run log or evaluation file in the repository. Software was developed with an AI coding assistant; all numbers come from runs on this machine.